



บทที่ 6 การวางแผนและการกำหนดเวลาโครงการ (Project Planning and Scheduling)

ผศ.วาสนา เหง้าเกษ



หัวข้อ

1. โครงสร้างการแบ่งงาน (Work Breakdown Structure - WBS)
2. แผนภูมิแกนต์ (Gantt Charts)
3. กรณีศึกษา

1. โครงสร้างการแบ่งงาน (Work Breakdown Structure - WBS)

โครงสร้างการแบ่งงาน (Work Breakdown Structure - WBS) คือการย่อยขอบเขตงานทั้งหมดของโครงการออกเป็นองค์ประกอบย่อยที่เล็กลงเพื่อให้จัดการได้ง่ายขึ้น WBS ไม่ใช่รายการกิจกรรมที่ต้องทำ แต่เป็นแผนผังที่เน้น “ผลลัพธ์ที่ส่งมอบได้” (Deliverables) ซึ่งเป็นผลิตภัณฑ์ บริการ หรือผลลัพธ์ที่จับต้องได้หรือวัดผลได้ที่เกิดจากการดำเนินงาน

วัตถุประสงค์หลักของการจัดทำ WBS มีดังต่อไปนี้

- 1) **การกำหนดขอบเขตงาน (Scope Definition)** กำหนดและจัดระเบียบขอบเขตของโครงการ (Project Scope) โดยการแบ่งขอบเขตทั้งหมดออกเป็นส่วนย่อยที่สามารถบริหารจัดการได้ ทำให้มั่นใจได้ว่างานที่จำเป็นทั้งหมดถูกระบุไว้ในแผนงานอย่างครบถ้วน และช่วยป้องกันปัญหา "ขอบเขตบานปลาย" (Scope Creep) ที่มักจะเกิดขึ้นในภายหลัง

- 
- 2) **การเป็นรากฐานการวางแผน (Foundation for Planning)** WBS คือแหล่งข้อมูลตั้งต้นหลักที่ใช้สำหรับกิจกรรมการวางแผนด้านอื่น ๆ เมื่อขอบเขตงานทั้งหมดถูกแบ่งย่อยเป็น Work Package ที่ละเอียดแล้ว จะสามารถนำไปประมาณการงบประมาณ เวลา และทรัพยากรที่จำเป็นสำหรับแต่ละงานได้อย่างแม่นยำมากขึ้น
 - 3) **การเพิ่มความเข้าใจที่เป็นหนึ่งเดียว (Unifying Understanding)** WBS ช่วยให้ทีมงานและผู้มีส่วนได้ส่วนเสียมีมุมมองที่เป็นหนึ่งเดียวเกี่ยวกับสิ่งที่ต้องส่งมอบ การแสดงภาพโครงสร้างในรูปแบบแผนผังลำดับชั้นที่ชัดเจนจะช่วยลดความสับสนและความไม่ชัดเจนในการสื่อสาร

หลักการสำคัญของ WBS

1) หลักการ 100% (The 100% Rule)

WBS ต้องครอบคลุมขอบเขตงานทั้งหมดของโครงการ 100% รวมถึงงานด้านการจัดการโครงการด้วย ผลรวมของงานในองค์ประกอบย่อย (child elements) จะต้องเท่ากับงานทั้งหมดในองค์ประกอบหลัก (parent element) เสมอ

การที่กำหนดให้งานในระดับย่อยจะต้องรวมกันได้ 100% ของงานในระดับบน ช่วยลดความเสี่ยงของการมี "งานที่ถูกซ่อน" (hidden work) ที่ไม่ได้ถูกระบุไว้แต่แรกและมักจะโผล่ขึ้นมาในภายหลัง และงานที่ไม่ได้ถูกระบุไว้ตั้งแต่ต้นนี้เองที่เป็นสาเหตุหลักที่ทำให้โครงการต้องล่าช้าและใช้งบประมาณเกิน

2) หลักการไม่ซ้ำซ้อน (Mutual Exclusivity)

งานในแต่ละองค์ประกอบของ WBS จะต้องไม่ทับซ้อนกันในแง่ของขอบเขต เพื่อป้องกันความสับสนในทีมงานและการทำงานซ้ำซ้อนที่ไม่มีประสิทธิภาพ

3) การเน้นผลลัพธ์ (Deliverables, not Actions)

WBS เน้นที่ "สิ่งที่ต้องส่งมอบ" (Deliverables) ไม่ใช่กิจกรรมหรือการกระทำ (Actions) ตัวอย่างเช่น ควรระบุว่า "ระบบล็อกอิน" (Login System) แทนที่จะเป็น "เขียนโค้ดระบบล็อกอิน" (Write Login System Code) หรือ "เว็บไซต์" (Website) แทนที่จะเป็น "ออกแบบเว็บไซต์" (Design Website)

4) ระดับความละเอียดที่เหมาะสม (Reasonable Level of Detail)

การย่อยงานควรทำไปจนถึงระดับที่งานย่อยนั้น (Work Package) สามารถมอบหมายให้ผู้รับผิดชอบคนเดียวหรือทีมเดียว สามารถประมาณการทรัพยากรและเวลาได้ และสามารถควบคุมความคืบหน้าได้ เกณฑ์ที่นิยมใช้คือ "กฎ 8/80" (8/80 Rule) ที่ระบุว่างานใน Work Package ควรใช้เวลาไม่ต่ำกว่า 8 ชั่วโมงและไม่เกิน 80 ชั่วโมง เพื่อให้งานไม่เล็กหรือใหญ่เกินไปจนยากต่อการบริหารจัดการ

องค์ประกอบและโครงสร้างของ WBS

WBS มีโครงสร้างแบบลำดับชั้น (Hierarchical) ที่มองเห็นได้ง่ายเหมือนแผนผังองค์กรหรือแผนภูมิต้นไม้ (Tree Diagram) โครงสร้างทั่วไปแบ่งออกเป็นระดับชั้นต่าง ๆ ดังนี้

ระดับ 1 (Level 1) คือเป้าหมายสูงสุดของโครงการ (Project Goal) หรือผลลัพธ์สุดท้ายที่ต้องส่งมอบ (Final Deliverable) ซึ่งเป็นชื่อของโครงการทั้งหมด

ระดับ 2 (Level 2) ประกอบด้วยผลลัพธ์หลักที่ต้องส่งมอบ (Major Deliverables) หรือเฟสงานหลักของโครงการ (Phases) สำหรับโครงการซอฟต์แวร์ อาจเป็น "การออกแบบ" "การพัฒนา" "การทดสอบ" เป็นต้น

ระดับ 3 และต่อไป (Level 3+) เป็นการย่อยผลลัพธ์หลักไปสู่ผลลัพธ์ย่อย (Sub-deliverables) ที่เป็นองค์ประกอบของผลลัพธ์หลักในระดับที่สูงขึ้น

ระดับต่ำสุด (Lowest Level) คือ Work Packages ซึ่งเป็นหน่วยงานย่อยที่เล็กที่สุดและละเอียดที่สุดที่สามารถประมาณการและควบคุมได้

ประเภทของ WBS

การจัดโครงสร้าง WBS สามารถทำได้หลายวิธี โดยวิธีที่นิยมใช้มีสองประเภทหลัก

WBS แบบเน้นผลลัพธ์ (Deliverable-Based WBS) จัดโครงสร้างตามผลิตภัณฑ์ บริการ หรือผลลัพธ์สุดท้ายที่ต้องส่งมอบจริง เหมาะสำหรับโครงการที่เน้นการสร้างผลิตภัณฑ์ที่จับต้องได้ เช่น การสร้างบ้าน

WBS แบบเน้นเฟส (Phase-Based WBS) จัดโครงสร้างตามขั้นตอนหรือวงจรชีวิตของโครงการ (Project Life Cycle) เช่น การวิเคราะห์ (Analysis) การออกแบบ (Design),การพัฒนา (Development) การทดสอบ (Testing) และการติดตั้ง (Launch) เหมาะสำหรับโครงการที่ทีมงานมีลักษณะข้ามสายงาน (Cross-functional) หรือสำหรับผู้เริ่มต้น เนื่องจากมีลำดับเชิงเส้นที่สอดคล้องกับกระบวนการทำงาน ทำให้สามารถเปลี่ยนไปเป็นแผนภูมิแกนต์ได้ง่าย

กระบวนการสร้าง WBS สำหรับโครงการซอฟต์แวร์

การสร้าง WBS ที่มีประสิทธิภาพเป็นกระบวนการที่ต้องทำซ้ำ ๆ และต้องได้รับความร่วมมือจากทีมงาน

ขั้นตอนที่ 1 กำหนดขอบเขตและวัตถุประสงค์ (Define Scope and Objectives) ก่อนที่จะเริ่มย่องาน ต้องเริ่มต้นด้วยการทบทวนเอกสารหลักของโครงการ เช่น Project Charter และ Scope Statement เพื่อให้เข้าใจเป้าหมาย ข้อจำกัด และข้อกำหนดต่าง ๆ ของโครงการอย่างถ่องแท้ การมีขอบเขตที่ชัดเจนเป็นรากฐานที่แข็งแกร่งสำหรับการสร้าง WBS ที่สมบูรณ์

ขั้นตอนที่ 2 ระบุผลลัพธ์หลัก (Identify Major Deliverables) ระดมสมองกับทีมงานเพื่อระบุผลลัพธ์หลักที่โครงการต้องสร้างขึ้น สำหรับโครงการพัฒนาแอปพลิเคชันมือถือ อาจมีผลลัพธ์หลักได้แก่ การออกแบบ UI การพัฒนา Backend หรือการตั้งค่าฐานข้อมูล

ขั้นตอนที่ 3 แยกย่อยงาน (Decomposition) เริ่มต้นกระบวนการย่อยผลลัพธ์หลักไปสู่ผลลัพธ์ย่อย และ Work Package ที่เป็นหน่วยงานที่เล็กที่สุด โดยยึดหลักการที่กล่าวมาข้างต้น กระบวนการนี้ควรทำอย่างเป็นระบบ จนกระทั่งแต่ละ Work Package สามารถมอบหมายให้ผู้รับผิดชอบคนเดียวได้

ขั้นตอนที่ 4 จัดทำ WBS Dictionary (Create WBS Dictionary) WBS Dictionary เป็นเอกสารที่ให้รายละเอียดเพิ่มเติมของแต่ละ Work Package ซึ่งจำเป็นต่อการดำเนินโครงการให้สำเร็จ รายละเอียดที่ควรมี เช่น คำอธิบายขอบเขตงาน ผู้รับผิดชอบ เกณฑ์การยอมรับ (Acceptance Criteria) งบประมาณ และความเสี่ยง WBS Dictionary ช่วยเพิ่มความชัดเจนและป้องกันความเข้าใจผิดในทีมงานได้อย่างมาก

ตัวอย่าง WBS สำหรับโครงการซอฟต์แวร์: การพัฒนาแอปพลิเคชันมือถือ

ตัวอย่าง WBS แบบเน้นเฟส (Phase-Based) สำหรับโครงการพัฒนาแอปพลิเคชัน E-commerce โดยแสดงในรูปแบบ Outline/Indented List ซึ่งเป็นรูปแบบที่อ่านง่าย

1.0 โครงการพัฒนาแอปพลิเคชัน E-commerce

1.1 เฟสการวางแผน (Planning Phase)

1.1.1 Project Charter และ Scope Statement

1.1.2 Work Breakdown Structure (WBS)

1.1.3 Project Plan และ Timeline

1.2 เฟสการออกแบบ (Design Phase)

1.2.1 UI/UX Design

1.2.1.1 Wireframes

1.2.1.2 Mockups และ Prototyping



1.2.2 Database Design

1.2.3 System Architecture Design

1.3 เฟสการพัฒนา (Development Phase)

1.3.1 Front-End Development (iOS/Android)

1.3.2 Back-End Development (API, Business Logic)

1.3.3 Database Implementation

1.4 เฟสการทดสอบ (Testing Phase)

1.4.1 Unit Testing

1.4.2 Integration Testing

1.4.3 User Acceptance Testing (UAT)

1.5 เฟสการเปิดตัว (Launch Phase)

1.5.1 App Store Submission (Apple App Store / Google Play Store)

1.5.2 Post-Launch Support & Maintenance

2. แผนภูมิแกนต์ (Gantt Charts)

แผนภูมิแกนต์ (Gantt Chart) คือแผนภูมิแท่งแนวนอน (Horizontal Bar Chart) ที่ใช้ในการแสดงกำหนดการของโครงการ ประกอบด้วยแกนแนวดิ่งสำหรับแสดงรายการงาน และแกนแนวนอนสำหรับแสดงระยะเวลา ถูกคิดค้นและเผยแพร่โดย Henry Gantt วิศวกรและที่ปรึกษาด้านการจัดการชาวอเมริกัน ในช่วงปี 1910-1915 องค์ประกอบหลักของแผนภูมิแกนต์ ได้แก่

- 1) **รายการงาน (Task List)** คือรายการของกิจกรรมหรืองานย่อยทั้งหมดที่ต้องดำเนินการในโครงการ โดยรายการงานเหล่านี้มักจะนำมาจาก Work Packages ซึ่งเป็นระดับต่ำสุดของ WBS
- 2) **เส้นเวลา (Timeline)** คือแกนแนวนอนที่อยู่ด้านบนของแผนภูมิ แสดงหน่วยเวลา เช่น วัน สัปดาห์ หรือ เดือน เส้นเวลานี้ทำให้ผู้ใช้สามารถเห็นภาพรวมของกำหนดการโครงการทั้งหมดได้
- 3) **แถบงาน (Gantt Bars)** คือแท่งแนวนอนที่ใช้แสดงระยะเวลา (Duration) ของแต่ละงาน จุดเริ่มต้นของแท่งคือวันเริ่มต้นของงาน และจุดสิ้นสุดคือวันสิ้นสุดของงาน แถบงานยังสามารถแสดงความคืบหน้าของงานได้ด้วยการแรเงาบางส่วน of แท่งเพื่อบ่งบอกเปอร์เซ็นต์ที่งานนั้นเสร็จสมบูรณ์แล้ว

- 4) **ความสัมพันธ์ระหว่างงาน (Dependencies)** คือเส้นเชื่อมโยงระหว่างแถบงานที่แสดงลำดับก่อนหลังในการทำงาน การแสดงความสัมพันธ์นี้เป็นสิ่งสำคัญที่ทำให้แผนภูมิแกนต์เป็นมากกว่ารายการงานทั่วไป
- 5) **เหตุการณ์สำคัญ (Milestones)** คือสัญลักษณ์พิเศษ เช่น **รูปข้าวหลามตัด (Diamonds)** ที่ใช้เพื่อแสดงเหตุการณ์สำคัญหรือจุดตรวจวัด หมายถึง "Milestone" หลักไมล์หรือจุดหมายสำคัญ ตัวอย่างของเหตุการณ์สำคัญ ได้แก่ การออกแบบ UI/UX (Design) ทำได้สำเร็จ ดังนั้น ผู้บริหารสังเกตรูป  ก็ทำให้ทราบคำว่า "ทำถึงจุดนี้แล้วหรือไม่" โดยไม่ต้องดูรายละเอียดงานย่อย
- 6) **ผู้รับผิดชอบ (Assignees)** การระบุว่าใครหรือทีมใดรับผิดชอบงานนั้น ๆ เป็นอีกหนึ่งองค์ประกอบสำคัญที่ช่วยให้การติดตามและการจัดการทรัพยากรมีความชัดเจน

ประเภทของความสัมพันธ์ระหว่างงาน (Task Dependencies)

ความสัมพันธ์ระหว่างงานเป็นหัวใจสำคัญที่ทำให้แผนภูมิแกนต์สามารถแสดงลำดับการทำงานที่ถูกต้องได้ มี 4 ประเภทหลักดังนี้

- **Finish-to-Start (FS)** งาน B จะเริ่มต้นได้ก็ต่อเมื่องาน A สิ้นสุดลงแล้ว เช่น การพัฒนาโค้ดต้องเสร็จสิ้นก่อนที่จะเริ่มทดสอบระบบ ความสัมพันธ์ประเภทนี้เป็นรูปแบบที่พบบ่อยที่สุด
- **Start-to-Start (SS)** งาน B สามารถเริ่มต้นได้เมื่อเริ่มต้นงาน A แล้ว เช่น การพัฒนาส่วนหน้าสามารถเริ่มต้นได้เมื่อการพัฒนาส่วนหลังเริ่มขึ้น
- **Finish-to-Finish (FF)** งาน B สามารถสิ้นสุดได้ก็ต่อเมื่องาน A สิ้นสุดลงแล้ว เช่น การจัดทำเอกสารโครงการจะเสร็จสมบูรณ์เมื่อการพัฒนาทั้งหมดเสร็จสมบูรณ์
- **Start-to-Finish (SF)** งาน B สามารถสิ้นสุดได้ก็ต่อเมื่องาน A เริ่มต้นแล้ว เป็นความสัมพันธ์ที่พบน้อยที่สุด

วัตถุประสงค์

- **การแสดงผลภาพรวมและการสื่อสาร** แผนภูมิแกนต์ให้มุมมองระดับสูง (High-level overview) ของโครงการทั้งหมด ทำให้ผู้มีส่วนได้ส่วนเสียทุกคนสามารถเข้าใจกำหนดการและไทม์ไลน์ของโครงการได้อย่างรวดเร็วในหน้าเดียว ซึ่งช่วยในการสื่อสารและสร้างความเข้าใจที่เป็นหนึ่งเดียวกัน
- **การจัดการทรัพยากร** แผนภูมิแกนต์ช่วยให้ผู้จัดการโครงการสามารถจัดสรรและวางแผนการใช้ทรัพยากร เช่น ทีมงาน ได้อย่างมีประสิทธิภาพ โดยเห็นภาพว่าใครกำลังทำงานอะไรและเมื่อไหร่ เพื่อป้องกันการทำงานที่ซ้ำซ้อนหรือภาระงานที่มากเกินไป
- **การติดตามและควบคุม** แผนภูมิแกนต์เป็นเครื่องมือที่ใช้ในการติดตามความคืบหน้าของโครงการเทียบกับแผนงาน (Baseline) และสามารถระบุงานที่ล่าช้าหรือจุดคอขวด (Bottlenecks) ได้อย่างรวดเร็ว ทำให้ผู้จัดการโครงการสามารถทำการปรับแผนได้ทัน

การสร้างแผนภูมิแกนต์จาก WBS

ขั้นตอนการแปลง WBS สู่ Gantt Chart

1) ระบุงาน

ขั้นตอนแรกคือการนำ Work Packages ซึ่งเป็นระดับต่ำสุดของ WBS มาใช้เป็นรายการงาน (Task List) ในแผนภูมิแกนต์ งานใน Work Packages ของ WBS ซึ่งมุ่งเน้นไปที่ผลลัพธ์ (Deliverable-based) จะถูกแปลงเป็นกิจกรรมหรืองาน (Task-based) ในแผนภูมิแกนต์

2) ประมาณการระยะเวลา

สำหรับแต่ละงานที่ระบุไว้ในขั้นตอนก่อนหน้านี้ ให้ทำการประมาณการระยะเวลาดที่ใช้ในการทำงานให้แล้วเสร็จ การประมาณการนี้ควรทำอย่างเป็นจริงและอาจรวมถึงการเผื่อเวลา (Buffer Time) สำหรับความไม่แน่นอนที่อาจเกิดขึ้น



3) กำหนดความสัมพันธ์

กำหนดความสัมพันธ์ระหว่างงาน (Dependencies) ว่างานใดต้องทำก่อนหรือหลังงานใด การระบุความสัมพันธ์นี้มีความสำคัญอย่างยิ่งต่อการกำหนดลำดับการทำงานที่ถูกต้อง

4) จัดลำดับและสร้างแผนภูมิ

นำรายการงาน ระยะเวลา และความสัมพันธ์ที่กำหนดไว้มาจัดเรียงบนไทม์ไลน์ โดยเริ่มจากวันเริ่มต้นของโครงการและกำหนดวันเริ่มต้นและวันสิ้นสุดของแต่ละงานอย่างเหมาะสม

3. กรณีศึกษา

โครงการพัฒนาแอปพลิเคชันส่งอาหาร (Food Delivery App)

1. Work Breakdown Structure (WBS)

แนวคิด WBS คือการมองภาพรวมของงานแล้ว "หั่น" (Decompose) ออกเป็นชั้นย่อย ๆ จนถึงระดับที่จัดการได้ (Work Package) โดยไม่สนใจเรื่องเวลา สนใจแค่ว่า "มีงานอะไรต้องทำบ้าง"

โครงสร้างจะเป็นลักษณะลำดับชั้น (Hierarchy) ดังนี้

1.0 การวางแผน (Planning)

1.1 รวบรวมความต้องการของผู้ใช้ (Requirement Gathering)

1.2 ศึกษาความเป็นไปได้และคู่แข่ง

1.3 จัดทำงบประมาณ



2.0 การออกแบบ (Design)

2.1 ออกแบบ User Interface (UI)

2.2 ออกแบบ User Experience (UX)

2.3 ออกแบบฐานข้อมูล (Database Schema)

3.0 การพัฒนา (Development)

3.1 พัฒนาฝั่งผู้ใช้ (Front-end: iOS/Android)

3.2 พัฒนาฝั่งระบบหลังบ้าน (Back-end)

3.3 เชื่อมต่อระบบชำระเงิน (Payment Gateway API)

4.0 การทดสอบ (Testing)

4.1 ทดสอบระบบภายใน (Unit Testing)

4.2 ทดสอบโดยผู้ใช้งานจริงกลุ่มเล็ก (Beta Testing)

4.3 แก้ไขบั๊ก (Bug Fixing)

5.0 การเปิดตัว (Launch)

5.1 นำแอปพลิเคชันขึ้นไปที่ Store (App Store/Play Store)

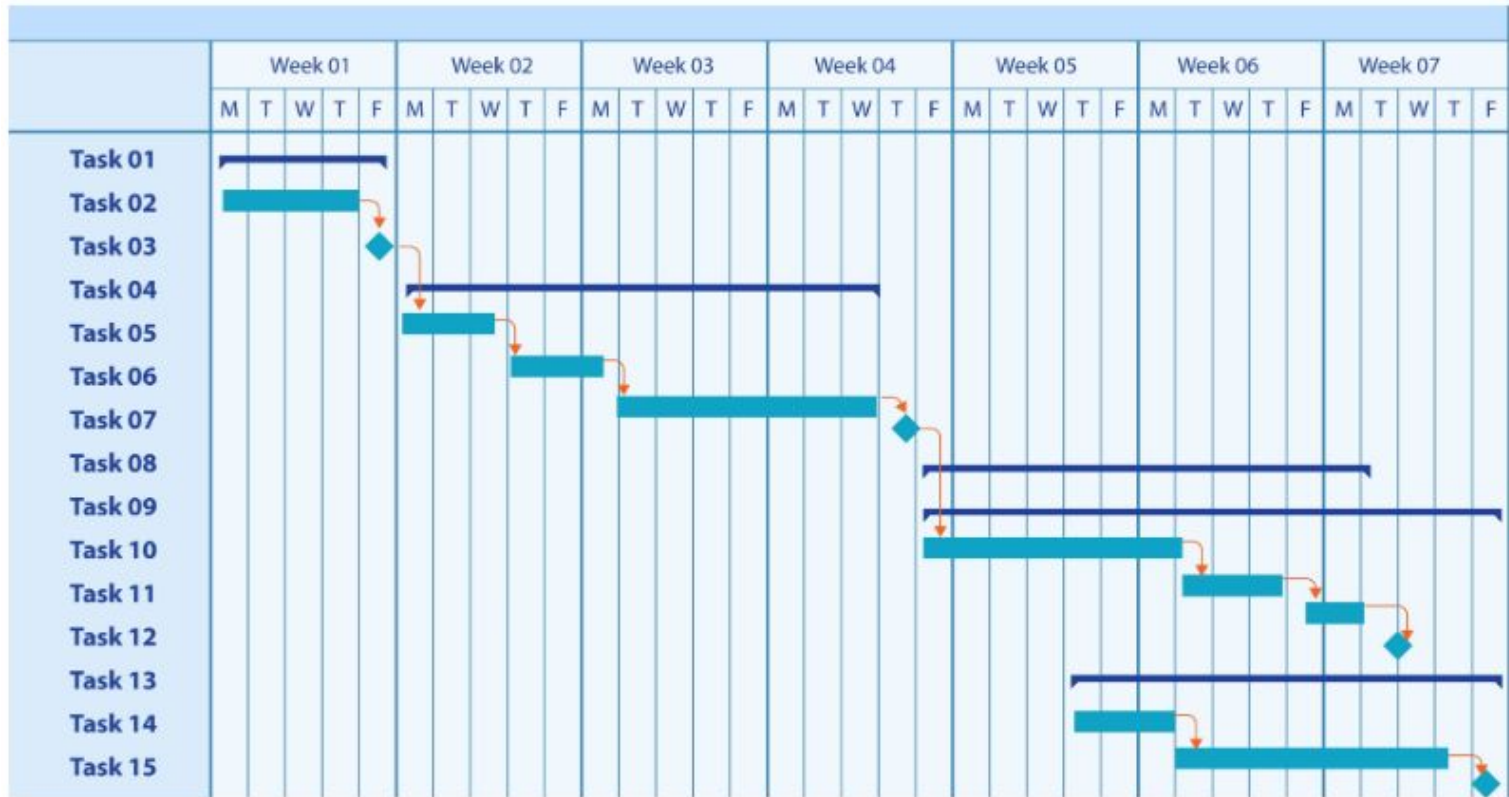
5.2 ทำการตลาดและโปรโมชัน

2. Gantt Charts

เมื่อได้รายการงานทั้งหมดจาก WBS แล้ว เราจะนำมาใส่ใน Gantt Chart เพื่อกำหนดว่า "งานไหนทำก่อน-หลัง" และ "ใช้เวลานานเท่าไร" ใน Gantt Chart จะเห็นความสัมพันธ์ (Dependencies) เช่น ต้องออกแบบ (2.0) ให้เสร็จก่อน จึงจะเริ่มพัฒนา (3.0) ได้

ตัวอย่างข้อมูลใน Gantt Chart ของโครงการ FoodDee:

งาน (Task)	ระยะเวลา (Duration)	เริ่มต้น (Start)	สิ้นสุด (Finish)	ต้องทำหลังจากงาน (Predecessors)
1.1 รวบรวมความต้องการ	1 สัปดาห์	1 ม.ค.	7 ม.ค.	-
2.1 ออกแบบ UI	2 สัปดาห์	8 ม.ค.	21 ม.ค.	1.1
3.1 พัฒนา Front-end	4 สัปดาห์	22 ม.ค.	18 ก.พ.	2.1
3.2 พัฒนา Back-end	4 สัปดาห์	22 ม.ค.	18 ก.พ.	2.1 (ทำพร้อม 3.1 ได้)
4.1 ทดสอบระบบ	2 สัปดาห์	19 ก.พ.	4 มี.ค.	3.1, 3.2
5.1 นำแอปลง Store	1 สัปดาห์	5 มี.ค.	11 มี.ค.	4.1



เครื่องมือวาด Gantt chart

Mermaid Live Editor <https://mermaid.live/>

หรือ VS Code Extension Mermaid Preview -> สร้างไฟล์นามสกุล .md (Markdown)

```
gantt
```

```
dateFormat YYYY-MM-DD
```

```
title โครงการพัฒนาแอปพลิเคชันสั่งอาหาร (FoodDee Gantt Chart)
```

```
section 1.0 การวางแผน
```

```
รวบรวมความต้องการ (Requirement) :done, task1, 2024-01-01, 1w
```

```
แผนงานอนุมัติ (Milestone Planning) :milestone, m1, after task1, 0d
```

```
section 2.0 การออกแบบ
```

```
ออกแบบ UI/UX (Design) :active, task2, after m1, 2w
```

```
แบบดีไซน์ผ่านการอนุมัติ (Milestone Design) :milestone, m2, after task2, 0d
```